

系統說明書與技術設計記錄

把一份 PowerPoint 投影片，直接轉換成一支有旁白配音、有字幕的課程影片 — 完整操作說明，以及三個核心技術子系統（字幕真實時間對齊、字幕斷句演算法、PowerPoint 自動化）的設計記錄。

最後更新 2026-08-06 目前版本 **v0.9.0** 後端測試 **195 passed**

文件目錄

第一部 系統說明書	P. 2
第二部 字幕真實起始時間對齊	P. 6
第三部 字幕斷句演算法	P. 9
第四部 PowerPoint 自動化的技術細節與已知限制	P. 12

這個系統是什麼

pptx2video 讀取一份 **.pptx** 簡報檔裡每一頁的「備忘稿」文字，依序完成 講稿解析 → 語音合成（TTS）→ 字幕產生 → 音訊插入投影片 → 匯出 MP4 五個階段，產出一支保留原生 PowerPoint 動畫與轉場、旁白配音與字幕時間都對齊實際播放畫面的課程影片。核心設計理念是「不用語音辨識（ASR）猜字幕」—— 備忘稿本身就是逐字稿，系統只需要把它對齊到語音實際播放的時間，準確度遠高於事後用 AI 辨識語音反推文字。

5

處理階段：解析 / 語音 / 字幕 / 插入音訊 / 匯出

Windows

需要 PowerPoint 才能插入音訊與匯出影片

190

自動化測試，涵蓋每個處理階段

系統說明書

這一部說明系統怎麼運作、怎麼操作、產出什麼、以及目前的已知限制 —— 給實際要使用這套工具的人看。

系統概觀

pptx2video 把一份 PowerPoint 投影片，轉換成一支有旁白配音、有字幕的課程影片。系統依序完成 **解析備忘稿** → **語音合成（TTS）** → **字幕產生** → **音訊插入投影片** → **匯出 MP4** 五個階段。

跟一般「用截圖 + 合成軟體拼接畫面」的做法不同，pptx2video 是實際驅動 PowerPoint 本身的「建立視訊」功能來匯出影片 —— 好處是投影片裡原本設計好的動畫、轉場效果、字型樣式都會完整保留在最終影片裡，不需要重新設計一套動畫系統；代價是這套系統目前只能在安裝了 PowerPoint 的 Windows 環境上運作。

關於逐字稿來源

目前唯一的講稿來源是投影片本身的「備忘稿」欄位 —— 上傳前只要把要唸的內容寫進每一頁的備忘稿即可，不需要另外準備逐字稿檔案，也不需要另外標記頁碼。

使用流程

01 安裝與準備

下載專案、建立 Python 虛擬環境、安裝相依套件；需要另外安裝 ffmpeg 並加入 PATH（產生字幕時量測音檔時長會用到）。

02 解析投影片

讀取 .pptx 每一頁的頁碼、標題與備忘稿文字，輸出成結構化的 JSON，供後續每個階段使用。

03 生成語音

用 Edge-TTS 把每頁備忘稿轉成 MP3，可調整配音、語速、音高；系統會同時記錄每個字實際播放的時間點，供字幕對齊使用。

04 產生字幕

把備忘稿拆成適合當一行字幕的片段，對齊到語音實際播放時間，輸出成一份 .srt 字幕檔（詳細設計見第三部）。

05 插入音訊並匯出 MP4

把生成好的語音插入對應投影片，再呼叫 PowerPoint 自動匯出成 MP4；字幕會在影片匯出完成後，依照真實畫面時間重新校正一次（詳細設計見第二部）。

06 一次到底 / 分段輸出 / 燒字幕

以上步驟可以拆開一步步跑，方便個別除錯，也可以合併成單一指令一次完成；影片太長時，另有工具可依換頁邊界切成多段，方便觀看與分享；分段的同時也能選擇把字幕直接燒進畫面（固定樣式的黑底白字字幕列），不依賴播放器是否支援字幕軌。

語音設定

語音由 **Edge-TTS** 提供，預設使用繁體中文語音「雲哲」，可依需求替換成 Edge-TTS 支援的其他語音（含多種語言與腔調）。

設定項目	可調範圍	說明
配音	Edge-TTS 語音庫	預設 zh-TW-YunJheNeural （繁中男聲），可替換成任何 Edge-TTS 支援的語音代碼
語速	可正可負	預設放慢 10%（ -10% ），正值加快、負值放慢
音高	可正可負	預設不調整（ +0Hz ）
重試次數	0 或正整數	語音生成失敗時（僅限暫時性網路/服務錯誤）自動重試的次數，預設 3 次

疑似漏講偵測

系統會自動比對每一頁「語音間隔」跟「這一頁自己的平均語速」，如果某段間隔明顯短於它涵蓋的文字量該有的時間，會主動提示「這裡可能被 TTS 引擎漏講了一段內容」，並標出建議覆聽的時間點——這是實際使用中真實發生過的問題（TTS 引擎悄悄跳過整段內容、且沒有任何錯誤訊息）之後加上的保護機制。

影片匯出設定

設定項目	可選值	說明
解析度	480 / 720（預設） / 1080 / 2160	對應 PowerPoint 匯出視窗的標準選項（垂直像素）
畫格率	整數，預設 30	匯出影片的每秒畫面格數
畫質	0–100，預設 85	對應 PowerPoint 匯出的編碼品質設定
無聲頁面停留秒數	預設 5.0 秒	封面/結尾等沒有配音的頁面要停留多久

產出與下載

完成後會產生兩個主要檔案：

MP4

課程影片（H.264 編碼，保留 PowerPoint 原生動畫與轉場）

SRT

獨立字幕檔，需搭配支援外掛字幕的播放器使用

與雲端服務型系統的差異

pptx2video 是在你自己電腦上執行的工具，不是雲端服務——產出的檔案就留在你指定的輸出資料夾裡，系統不會自動幫你清除，也沒有保留期限的限制；相對地，也需要你自己管理輸出資料夾的空間與備份。

進階功能

A 只重新生成特定頁面

某一頁需要修改語音、或收到疑似漏講提示需要重跑時，可以只指定該頁重新生成，不需要整份長講稿重新跑一次（長講稿完整重跑可能超過一小時）。

B 字幕/播放時間校準

長影片下，字幕時間量測可能出現一個極小、隨播放時間累積的系統性偏差；系統提供自動估算與手動校準兩種方式取得校正係數，套用後可把偏差壓到次秒等級。詳見第二部。

C 依換頁邊界分段輸出

影片太長（例如 20 頁講稿匯出後長達 2~3 小時）時，可以把已匯出的單一 MP4 事後切成多段，切點會精準落在換頁的地方，不會切在講稿講到一半；字幕也會同步切出對應的分段版本。

系統需求

項目	需求
作業系統	Windows（音訊插入與 MP4 匯出功能需要）
PowerPoint	需已安裝，供系統自動化插入音訊、匯出 MP4
Python	3.9 或以上
網路	語音生成需要能連線到 Edge-TTS 服務
ffmpeg	產生字幕（量測音檔時長）需要，須另外安裝並加入 PATH

已知限制（摘要）

限制	說明
現場放映模式	插入音訊後，在「投影片放映」現場播放模式下仍需點擊音符圖示才會出聲；已確認 不影響 匯出的 MP4 影片
字幕時間校準	真實起始時間對齊模式量出來的時間，帶有一個環境相依的系統性偏差，長影片建議先校準（見第二部）
極短獨立字幕行	備忘稿裡自成一語的極短句子，目前會變成一段顯示時間很短的獨立字幕行，尚待優化
純英文字幕行寬	目前的行寬設定是針對中文字幕調校，大段純英文內容換行可能不夠自然

五階段管線總覽

下圖是整個系統的資料流向：從輸入的 **.pptx** 開始，到最終輸出 MP4 與 SRT 字幕檔為止，每個階段各自的輸入輸出如下。

① 解析 (Parse)

輸入：.pptx 檔案 → 輸出：每頁的頁碼、標題、備忘稿文字 (JSON)

② 語音合成 (TTS)

輸入：每頁備忘稿文字 → 輸出：每頁 MP3 音檔 + 逐字時間資料 (供字幕對齊使用)

③ 字幕產生 (Subtitles)

輸入：備忘稿文字 + 逐字時間資料 → 輸出：一份完整的 .srt 字幕檔

④ 插入音訊 (Insert Audio)

輸入：.pptx + 每頁 MP3 → 輸出：已插入配音的 .pptx

⑤ 匯出影片 (Export Video)

輸入：已插入配音的 .pptx → 輸出：.mp4 課程影片 (若同時要求字幕，會在此階段結束後，用實際匯出的畫面重新校正字幕時間，見第二部)

為什麼字幕要分兩階段產生

字幕的時間軸有兩種算法：影片還沒匯出完成前，只能靠「把每頁音檔的長度加總」去**預測**每一頁在最終影片裡的位置；等影片真正匯出完成後，系統會改用**比對實際匯出畫面的音軌**去量出每一頁真正的起始時間，取代原本的預測值，準確度更高，尤其是長講稿。完整設計思路與真實案例，見下一部。

字幕真實起始時間對齊

這是 pptx2video 因為「真的驅動 PowerPoint 匯出影片」而必須解決、也是這套系統比較獨特的技術問題 —— 字幕的時間軸，要怎麼跟 PowerPoint 實際匯出出來的畫面對得上。

問題從哪裡來

PowerPoint 的「建立視訊」匯出功能是一個黑盒子 —— 你把已插入配音的投影片交給它，它吐出一支 MP4，但每一頁在最終影片裡實際停留多久、切換點確切落在哪一秒，PowerPoint 本身並不會告訴你。系統一開始用的是最直覺的做法：把每一頁音檔的實際長度依序加總，猜測每一頁在最終影片裡的起始時間（稱為「**預測模式**」）。

這個假設在小規模測試簡報上驗證過偏差在 0.2 秒內；但在一份真實的長講稿（2 小時 40 分、20 頁）上重新測試後，發現預測時間軸會逐頁累積漂移，到後面差到好幾秒，而且不是單純等比例的關係，沒辦法用一個固定倍率去校正。

解法：真的去匯出好的影片裡量測

與其猜測，不如直接測量。系統改成：等 PowerPoint 真正把影片匯出完成後，把這支影片的音軌抽出來，拿每一頁自己的 配音去跟這整條音軌做音訊比對（互相關分析），量出這一頁的旁白**真正**是在影片的第幾秒開始播放的 —— 這稱為「**真實起始時間模式**」，取代原本的預測值。只有指令裡同時要求「輸出字幕」跟「匯出影片」時，系統才會等影片 真正匯出完成後才寫出字幕，確保寫出來的是這個比較準的版本。

模式	觸發時機	準確度
預測模式	只要求字幕、還沒匯出影片	短講稿夠用；長講稿可能累積到數秒偏差
真實起始時間模式	字幕與匯出影片在同一指令中一起要求	比對實際畫面音軌，準確度高，仍可能需要下方的系統性偏差校正

量測本身也有一個系統性偏差

改用真實測量後，準確度大幅提升，但在同一份 2 小時 40 分鐘的長講稿上，逐頁用播放器精確核對真實時間後，發現量測結果本身還帶有一個很小、但跟已播放時間成正比的系統性偏差（整份講稿累積到 12 秒等級）。已經排查並排除的可能成因包括「PowerPoint 匯出時加速了音訊」「匯出檔案本身音畫不同步」「量測過程中的重取樣誤差」——確切是比對演算法內部哪個環節造成的，目前尚未定位到程式碼層級，但已經確認**套用一個簡單的乘數修正**就能把這個偏差壓到接近誤差範圍內。

這不是一個通用常數

這個修正係數是經驗校準值，換一份講稿、換一台機器，理論上都需要重新校準——不是寫死在系統裡就能一勞永逸的設定。

真實校準案例

以下是兩次獨立、在不同機器與不同講稿上做的真實校準結果：

案例	講稿規模	校準值	校正後殘差
案例一	20 頁，約 2 小時 40 分	$\times 1.00121$	RMS 0.27 秒，全片最大 0.53 秒
案例二（獨立驗證）	18 頁，不同機器	$\times 1.001221$	RMS 0.170 秒，最大 0.312 秒

兩次獨立校準的係數只差在小數點第五位，幾乎一致——這是支持「這個偏差可能是 PowerPoint 匯出流程本身的普遍特性、而非單一機器的本地因素」這個假說的實際證據，但目前仍只有兩個資料點，還不足以直接下定論、改成系統內建的保守預設值。

怎麼校準

手動校準

用播放器精確核對幾個真實時間點，換來獨立人耳驗證過的校準值，適合準確度要求很高的場合，但比較費工。

自動估算（推薦先試）

系統自動抽樣比對匯出好的影片，回歸算出建議校準值，不需要人耳確認，適合快速取得一個堪用的校正係數。

如果影片被切成多段，字幕還準嗎

影片太長需要事後分段時（見第一部「依換頁邊界分段輸出」），系統會重用跟切影片時完全相同的切點時間戳去裁切、歸零字幕檔，而不是另外重新量一次——這樣才能保證每一段影片跟它自己的字幕檔對「時間 0 秒」的認定完全一致，不會因為兩次獨立量測結果有些微差異，而讓分段後的字幕跟畫面對不齊。

設計原則

凡是需要「同一份時間量測結果被用在兩個地方」的情境（例如同時要切影片、也要切字幕），系統一律選擇**共用同一次量測**，而不是讓兩邊各自獨立重新測量——避免因為量測本身存在的微小誤差，反而在兩個本該一致的產物之間製造出新的不一致。

小結

字幕真實起始時間對齊這條技術路線的核心思路是：**能實測就不要用猜的**（真實起始時間模式取代預測模式），**猜不準的地方明確測量並修正**（系統性偏差的經驗校準），**同一個答案只算一次、到處共用**（分段輸出重用同一次量測）。這三個原則背後都是被真實的長講稿逼出來的設計決策——沒有一個是憑空預先設計好的，都是實際踩過偏差之後才加上去的。

字幕斷句演算法

備忘稿是一整段連續的文字，要怎麼切成一行一行、讀起來自然、又不會切壞詞語的字幕，是另一個獨立的技術問題——這一部只講「文字要在哪裡斷」，跟第二部「字幕的時間戳怎麼來」是分開的兩件事。

設計目標

每一段字幕只顯示一行；斷句的位置要落在讀起來自然的地方（標點符號、詞語邊界），而不是隨機或硬切在詞語中間。另外，備忘稿本身是使用者自己在 PowerPoint 裡打字/貼上的內容，常常夾帶多餘的空白、標點使用不一致，斷句前也需要先把這些雜訊清乾淨。

切法：以「顯示寬度」為單位，先估行數再平均分配

系統不是用單純的字數上限，而是用**顯示寬度**：中文全形字算 2、英文數字等半形字算 1，預設每行上限是 36（相當於 18 個中文全形字）——這樣設計是因為中英文混排時，如果只算字數，一行英文跟一行中文讀起來的「份量」會差很多，用顯示寬度換算後兩者比較接近。

1 先依句子結束的標點切開

用句號、問號等主要標點，把整段備忘稿切成一句一句的候選單元。

2 貪婪地把短句塞進同一行

把相鄰的短句盡量塞進同一行字幕，直到超過寬度上限為止——避免「好。」這種極短句子自己獨占一行。

3 整段太寬時，退回用逗號、頓號等次要標點再切一次

單一句子本身太長時，先嘗試在子句層級的標點處再切開，一樣盡量塞滿每一行。

4 完全沒有標點可切時，用中文分詞找邊界，絕不切在詞語中間

用中文分詞工具（jieba）找出詞語邊界，只在詞語與詞語之間切，不會把一個詞攔腰切開；真實內容測試中，字元層級硬切 15 次裡 5 次切壞詞語，改用分詞邊界後 16 次裡 0 次。

5 算出最少需要幾行後，重新分配讓每行寬度盡量平均

單純貪婪塞滿容易在句尾留下一段很短、讀起來突兀的殘段（例如把「控制核心」硬切成「…控制」／「核心」兩段）；系統會先算出這段話最少需要幾行，再在同樣行數下重新分配，讓每一行的份量盡量平均。

額外的保護機制

情境	處理方式
數字裡的小數點/千分位（如 3.3V、1,000,000）	不會被誤判成句子結束的標點而切開，也不會在重新排版時被誤刪
段落邊界	備忘稿裡的段落永遠是硬邊界，字幕不會跨段落合併——段落是使用者自己刻意安排的停頓節奏，系統不會替使用者決定要不要合併
句尾多餘標點	顯示前會清除逗號、句號等純節奏性標點，但保留問號、驚嘆號（它們承載語氣，不會被清掉）
多餘空白	兩個中文字之間的空白（多半是打字/貼上時不小心留下的）會整個清除；其他位置的空白會收斂成一個

真實案例：避免把複合詞尾巴留成孤兒行

單純貪婪塞滿・較差

而是它扮演了整個 Embedded System 的控制

核心

「控制核心」被硬生生拆成兩段字幕，第二段只剩「核心」兩個字，畫面上一閃即過。

重新平均分配・目前作法

而是它扮演了整個 Embedded

System 的控制核心

兩段字幕份量接近，且「控制核心」這個詞完整留在同一段。

目前尚未處理的情境

引號內文字目前沒有特別保護

如果一段話裡包含引號，且引號的收尾符號後面沒有緊接逗號或句號，目前的斷句邏輯不會把引號收尾當成一個天然斷點，可能繼續往後找標點或依詞語邊界硬切，理論上可能把引號內的一段話拆成兩段字幕。這是已知、尚待補上的保護機制，優先度較高，因為已經確認是結構性的缺口，不是理論上的臆測。

極短獨立段落

備忘稿裡自成一段的極短句子（例如「例如：」），因為段落是硬邊界、不能跨段合併，目前會變成一段顯示時間很短的獨立字幕行，尚待決定要放寬段落合併規則、還是在時間軸層級補上最短顯示時間。

字幕怎麼跟畫面結合

目前系統輸出的 `.srt` 是一份**獨立字幕檔**，不會燒錄（烙印）進 MP4 畫面裡——也就是說匯出的影片本身是乾淨、沒有字幕的，字幕需要另外用支援外掛字幕的播放器載入播放。這個設計的好處是同一支影片可以搭配不同語言/版本的字幕檔重複使用，且不會因為字幕燒錄而永久性地影響畫面；如果需要「影片打開就直接有字幕」，需要額外用影片處理工具把字幕燒錄進畫面，目前不是系統自動完成的一環。

小結

字幕斷句演算法的核心思路是：**優先在自然的地方斷句，而不是硬性依字數切**（標點分層 + 分詞邊界保護），**避免份量不均的孤兒行**（先算最少行數，再平均分配），**已知的坑優先修，未知的坑誠實列出來**（數字、段落邊界已經處理，引號保護跟極短段落仍是待辦事項）。

PowerPoint 自動化的技術細節與已知限制

因為系統是真的驅動 PowerPoint 完成插入音訊與匯出影片，過程中累積了一些「只能靠實測才能確認、文件完全查不到」的行為，記錄在這裡，讓使用系統的人了解為什麼系統會這樣設計。

看不出效果、但拿掉會壞掉的設定

插入配音時，系統會設定一個 PowerPoint 舊版 API 裡的播放旗標。這個設定在 PowerPoint 編輯畫面上完全看不出任何差異，投影片的播放設定顯示的內容跟沒設定時一模一樣——但實際測試發現，**拿掉這個設定，匯出的影片就會完全沒有聲音，而且每一頁的停留時間會變回固定的 5 秒，不會依照配音長度調整**。目前系統固定會設定這個旗標，即使底層原理尚未完全查證清楚。

匯出狀態的安全網

PowerPoint 判斷匯出「是否完成」的狀態值，是依照官方文件的假設，並未在所有 PowerPoint 版本上逐一驗證過。為了降低誤判風險，系統即使收到「匯出完成」的回報，仍然會額外檢查輸出檔案是否真的存在、且內容非空，才會真正視為成功——避免因為狀態值誤判而讓使用者以為影片已經匯出好，實際上檔案是空的或根本不存在。

轉場時間不需要額外處理

PowerPoint 的「建立視訊」匯出功能，在沒有手動錄製播放時間的情況下，本來就會自動依嵌入媒體（也就是插入的配音）的時長，決定每一頁該停留多久——系統不需要額外計算或設定每頁的轉場秒數。

現場放映模式仍需點擊音符圖示

透過自動化插入的配音，在 PowerPoint 編輯畫面上的播放設定固定顯示「按一下時」，跟手動用滑鼠插入的效果不同；這代表如果直接用「投影片放映」模式現場簡報，需要多點一下音符圖示才會出聲。

已確認不影響匯出的 MP4 影片

這個限制只影響「現場播放」情境，匯出的 MP4 影片本身完全不受影響——因為系統的主要用途是產出可發佈的課程影片檔案，不是用來現場簡報，所以目前刻意優先確保匯出正確，暫不處理這個限制。

為什麼失敗時不自動重試

語音生成走網路 API，遇到暫時性的網路/服務錯誤時，系統會自動重試；但插入配音、匯出影片這類需要驅動 PowerPoint 的操作，**刻意不做自動重試**——因為重試前如果沒有先確保前一次啟動的 PowerPoint 已經完全關閉，貿然重試反而可能在背景留下多個殘留的 PowerPoint 行程，風險比「失敗後停下來讓使用者自己確認」更高。

小結

這一部記錄的每一項行為，都不是預先設計好的，而是**先看到匯出結果不對，才回頭找出原因、加上對應的處理方式**。這也是為什麼這份文件把它們個別記錄下來，而不是只留一句「已修正」——讓之後任何人（包含使用系統的人）在遇到類似的匯出異常時，能先想到「這是不是也是 PowerPoint 本身的某個特性」，而不是從頭排查。